

Projet de Fin d'Année: Cahier de Charges

**Conception et Développement d'un Moteur de
Génération de Séquences Peptidiques Intelligent par
un Algorithme Heuristique (DFS) et principes de l'IA
classique.**

Réalisé par : Mhamdi Mohamed Habib & Ghofrane Ben Boubaker

Sommaire

I.Présentation Générale du projet	3
1.1: Contexte du projet, inspiration biologique	3
1.2: Problématique: Le défi combinatoire	3
1.3: Solution Proposée	3
1.4 Heuristique Hybride : Greedy avec Pénalité de Répétition	4
1.5 Stratégie d'élagage :	4
1.6 Validation par Similarité :	4
1.7 Architecture Technique	4
II. ANALYSE DES BESOINS	5
2.1 Besoins Fonctionnels	5
2.2 Besoins Non Fonctionnels	6
III.Conception et Architecture	7
3.1 Architecture globale:	7
3.2 Diagramme de séquence :	8
3.3 Modélisation des Données : Schéma Relationnel	9
3.4 diagramme de cas d'utilisation :	10
IV Environnement de réalisation	12
4.1 Environnement Logiciel (Stack Technique)	12
4.2 Environnement Matériel & Tests	12
V.Panification, Méthodologie Scrum:	13
5.1 Méthodologie Agile (Scrum)	13
5.2 Définition des sprints	13

I.PRESENTATION GENERALE DU PROJET

1.1: Contexte du projet, inspiration biologique

Des études scientifiques récentes, notamment par l'université de Singapour, ont exploré le potentiel de certains peptides dans l'amélioration de fonctions cognitives liées au vieillissement, ces travaux constituent l'une des sources d'inspiration pour notre projet.

Ce projet aspire de concevoir un système intelligent capable de générer des séquences peptidiques qui imitent des profils physico-chimiques des protéines naturelles, facilitant ainsi la découverte de nouveaux candidats pour divers buts.

1.2: Problématique: Le défi combinatoire

Il existe 20 acides aminés protéinogènes qui sont les briques de base pour la vie, ainsi une séquence de longueur n possède 20^n combinaisons possibles,

Pour un peptide de 20 acides aminés par exemple,

l'espace de recherche atteint : 104,857,600,000,000,000,000,000 combinaisons

l'objectif est de trouver une solution satisfaisant toutes les contraintes mathématiques et physico-chimiques dans cet espace massif.

Parmi les contraintes mathématiques on cite :

- La masse moléculaire cible (en Da)
- L'hydrophobicité (Echelle de KyteDoolittle)
- L'affinité de liaison (Somme de polarité + charge)
- La stabilité (Indice Aliphatique)
- La faisabilité Biologique (Repliement, extrémités chargées)

1.3: Solution Proposée

Pour répondre à la problématique ci-dessus, nous proposons un Agent IA Classique qui se repose sur les recherches heuristiques, il utilise un algorithme DFS optimisé par une stratégie de Branch and Bound, Sliding Window pour éviter l'encombrement stérique et les extrémités chargées, algorithme de Chou-fasman pour promouvoir le repliement de la peptide.

Un Point Majeur de notre projet est qu'il suit les pas de XAI « Explainable AI » résumée par le principe « Making AI justify its own choices »

En effet, L'agent ne se contente pas de retourner simplement une séquence finale : il génère un historique complet de décisions structurés lors de l'exécution et la recherche des acides aminés (Step, Action, Level, Last_Amino_Acid_Added, Pruning_Reason) d'où chaque action d'ajout ou d'élagage est tracée.

Cet historique complet sera envoyé et visualisé au niveau de l'interface utilisateur transformant ainsi un algorithme précédemment silencieux, en une expérience visuelle et même éducative.

1.4 Heuristique Hybride : Greedy avec Pénalité de Répétition

Pour naviguer l'espace de recherche immense, l'agent implémente une stratégie Greedy (Goulue) : à chaque niveau de l'arbre DFS on priorise l'acide aminé qui va maximiser toutes les contraintes biochimiques et mathématiques, on affecte un score à tous les acides aminés selon leurs contributions aux contraintes, celui ayant le score le plus haut sera ajouté, malgré son efficacité, cette approche pose un autre problème : l'agent peut trouver une séquence très répétitive, ainsi on ajoute une pénalité de répétition d'un même acide aminé, soustrayant ainsi des points de son score pour promouvoir la diversité et l'entropie de la séquence.

1.5 Stratégie d'élagage :

Le cœur de la solution réside dans l'élagage Branch and Bound qui permet d'éviter milliards de branches éventuellement impossibles dès le début, cette approche justifiera l'abandon d'une branche quelconque dès qu'au moins une contrainte est violée.

1.6 Validation par Similarité :

La séquence générée sera validée en un processus à deux étapes :

En premier lieu nous comparons les contraintes mathématiques comme l'hydrophobicité, la masse moléculaire, la stabilité et l'affinité de liaison aux celles dictées par l'utilisateur

En second lieu nous calculons l'aire sous les graphes des profils de distribution des propriétés physico-chimiques le long de la séquence qui permet de quantifier la répartition spatiale des propriétés offrant une mesure de comparaison plus robuste.

1.7 Architecture Technique

Le Project s'appuie sur une stack web robuste :

- Un Moteur de calcul en Python qui implémente les heuristiques et les fonctions mathématiques et qui fournit une API rest avec flask.

- Orchestrateur et sécurité à l'aide de SpringBoot qui gère la persistance des données MySQL et la sécurisation par spring security et JWT
- Visualisation et interfaçage par Angular.

II. ANALYSE DES BESOINS

2.1 Besoins Fonctionnels

Décrivent les actions que le système doit être capable d'exécuter :

Numéro du besoin	Fonctionnalité	Description
1	Paramétrage de l'Agent	Saisie des contraintes et choisir s'il faut respecter les contraintes biologiques (repliement..) ou non.
2	DFS Greedy avec élagage	Rechercher en profondeur dans l'espace des acides aminés priorisant celui qui donne le plus gain immédiat, Arrêt des branches invalides avec sauvegarde de la justification
3	Pénalité de Répétition	Une heuristique qui empêche la répétition excessive d'un même acide aminé pour assurer la diversité peptidique
4	Validation par calculs et comparaison des AUC(Area Under Curve) des graphes de profils des propriétés	Calcul des propriétés de la séquence générée et calcul de similarité via la méthode np.trapz de la bibliothèque numpy de python pour les graphes de profils des propriétés le long de la séquence
5	Génération des graphes des profils des propriétés	Le module Python calcule les tableaux des points et les envoie vers l'interface pour qu'elle puisse afficher ces graphes, les graphes de

		la séquence à comparer avec la séquence obtenue seront en chevauchement
6	Sécurisation par JWT et spring Security	Protection des API par une authentification stateless grâce à spring Security
7	Visualisation des traces de l'agent	Le processus DFS sera visualisé comme un graphe grâce à l'historique complet qui est envoyé du module python.
8	Sauvegarder une séquence	Une fois générée, l'utilisateur a l'option de sauvegarder le résultat ainsi que l'historique de l'algorithme dans la base de données
9	Consulter les séquences associés à l'utilisateur	Une page sera consacré afficher toutes les séquences sauvegardés par l'utilisateur ainsi que les informations qui les sont associés
10	Gestion de séquences	L'utilisateur peut chercher un séquence dans la base de données a l'option de la supprimer

2.2 Besoins Non Fonctionnels

- Performances et optimisation :
 - L'agent doit explorer des milliers vu milliards de nœuds et retourner une séquence valide ainsi que son historique avec un délai de temps acceptable, il faut noter que bien qu'il n'y ait pas de limite pour la longueur de la séquence à générer, une longueur de séquence très large va naturellement causer un temps d'exécution plus long, pour empêcher les temps d'exécution trop long, on a imposé une limite au

nombre maximal de nœuds traversés par le DFS, une configuration utilisateur qui entraîne le dépassement de cette limite engendra un message d'erreur et recommandera d'assouplir les contraintes.

- Sécurité des données :
 - Chaque requête vers le module de calcul python doit être validée par un Token JWT pour empêcher tout accès non autorisé.
- Expérience utilisateur (UX) :
 - Navigation fluide entre les pages, un design moderne et réactif
- Scalabilité :
 - L'architecture de l'application doit permettre de gérer des calculs lourds sans qu'elle se plante.
 - Un autre point majeur de notre projet est qu'on a optimisé l'envoi de l'historique complet de l'agent on a traité ce flux de données par « streaming » en effet on envoie partie par partie les étapes à fur et à mesure de leur génération grâce à la méthode « yield » de flask pour garantir la fluidité de l'application.

III.CONCEPTION ET ARCHITECTURE

3.1 Architecture globale:

L'architecture repose sur une séparation stricte des préoccupations pour garantir la scalabilité et la maintenance de système .

Couche présentation (Angular) : une application qui gère l'état complexe des formulaires de contraintes biochimiques .Elle utilise des composants réactifs pour afficher en temps réel les graphiques de distribution physico-chimique et l'arborescence de recherche de l'IA.

Couche d'orchestration (SpringBoot) : ce module agit comme un API Gateway et un gestionnaire de persistance. Il sécurise les échanges via JWT(JSON Web Tokens) et expose des points de terminaison REST pour le frontend .il délègue les tâches lourdes de bio-informatique au moteur python via des appels .

Moteur de calcul Intelligent(Python/Flask) : Ce service est encapsulé dans un environnement Flask pour sa légèreté .il héberge l'algorithme DFS optimisé. L'utilisation de python est justifiée par la richesse de ses bibliothèques scientifiques pour manipuler les échelles de Kyte-Doolittle et les indices aliphatiques.

3.2 Diagramme de séquence :

Cette section détaille le protocole de communication asynchrone entre le client Angular et l'orchestrateur Spring Boot et le moteur d'interface Python .Le diagramme de séquence est conçu pour garantir la réactivité de l'interface tout en traitant des calculs intensifs.

Description de flux Opérationnel :

1-Phase d'initialisation : l'utilisateur soumet les paramètres biochimiques (Masse,hydrophobicité,stabilité...)via un formulaire réactif.

2- Interception & Sécurisation : L'API Gateway (Spring Boot) valide le jeton JWT de la session et persiste la demande en base de données pour assurer la traçabilité.

3-Délégation Algorithmique : Spring Boot effectue un appel REST POST vers le micro-service Flask. Ce dernier reçoit un objet JSON contenant les contraintes mathématiques cibles.

4- Exploration & Logging XAI : Le moteur Python exécute l'algorithme DFS. À chaque itération (ajout d'un acide aminé ou élagage), une trace structurée est générée (Step, Action, Pruning_Reason).

5- Agrégation des Résultats : Une fois la séquence optimale identifiée, le moteur Python renvoie un "Payload" complet contenant la séquence et l'historique décisionnel.

6-Finalisation : Spring Boot met à jour l'état de la transaction dans MySQL et transmet les données finales au client Angular pour la visualisation des profils de distribution.

7-sauvegarder la séquence :l'utilisateur peut enregistrer la séquence.

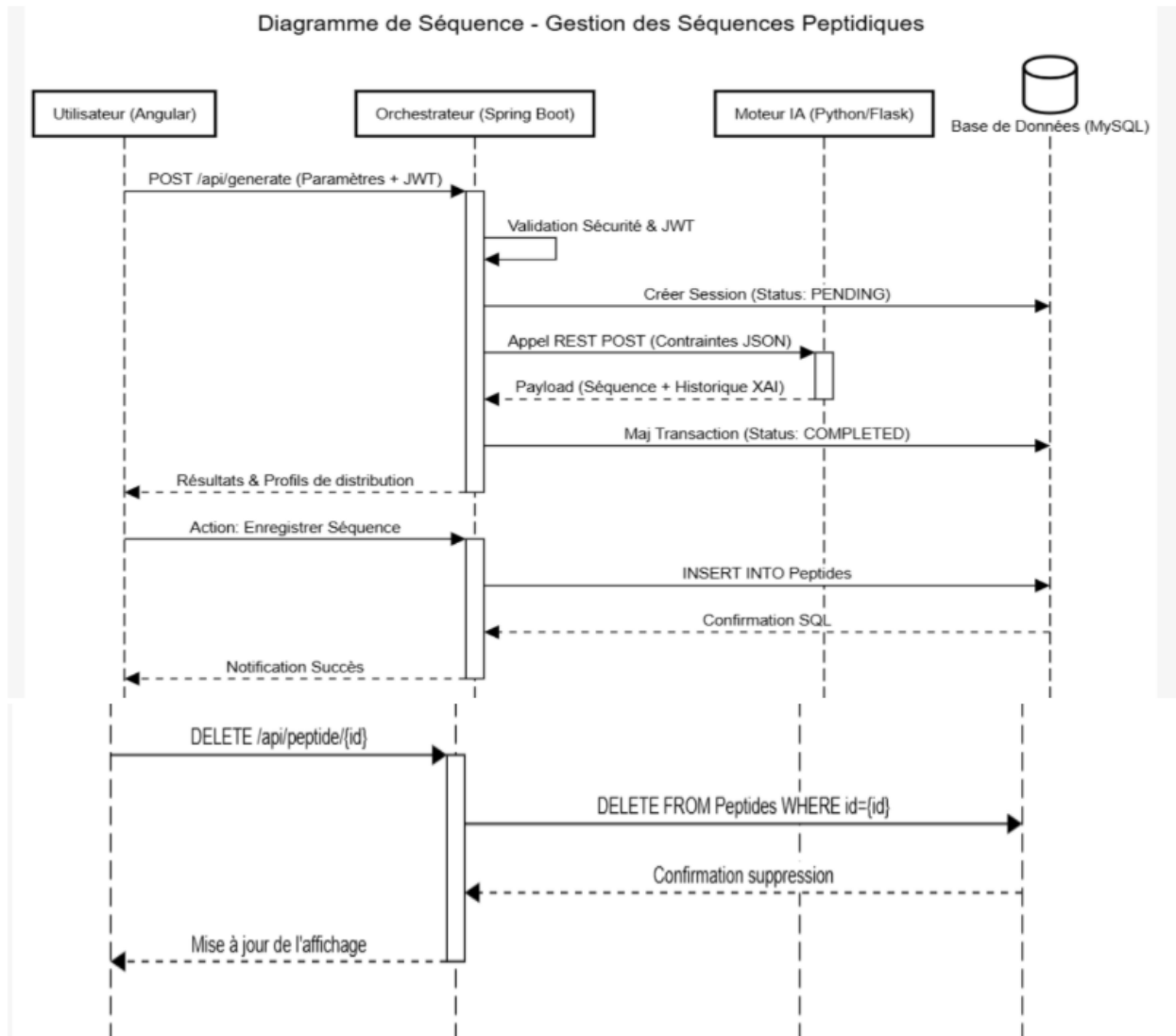


Figure 1: Diagramme de séquence

3.3 Modélisation des Données : Schéma Relationnel :

La conception de la base de données MySql suit les règles de normalisation pour garantir l'intégrité des données et la performance des requêtes analytiques sur les séquences générées.

- Les entités principales :

Entité **User** : Gère le contrôle de l'accès . Elle stocke les identifiants uniques et les informations d'authentification sécurisées pour Spring Security.

Entité **PeptideSequence** :représente le résultat final de la génération .Elle contient la chaine primaire (séquence d'AA)et les métadonnées de performance .

Entité **BiochemicalConstraints** : Stocke les paramètres d'entrée définis par l'utilisateur pour une génération donnée (Masse cible, Echelle Kyte-Doolittle, Indice Aliphatique).

Entité **GenerationTrace** (Cœur de la XAI) : Cette table est critique pour l'aspect Explainable AI. Elle enregistre chaque étape du parcours de l'arbre DFS :

-step_index : Ordre chronologique de l'action.

-action_type : « Add », « Prune » ou « Done »

-pruning_reason : Motif technique de l'abandon d'une branche (comme Violation de la fenêtre de glissement ou défaut de repliement Chou-Fasman).

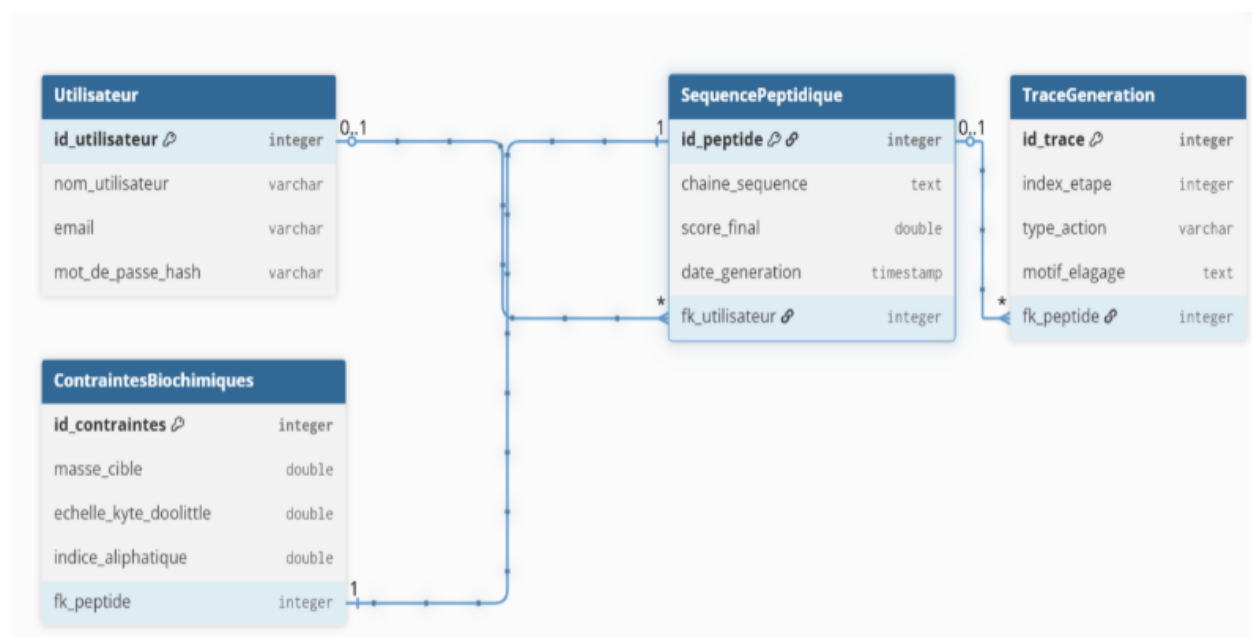


Figure 2:Modèle Physique de données

3.4 diagramme de cas d'utilisation :

Le diagramme de cas d'utilisation illustre les interactions entre les acteurs externes et les fonctionnalités offertes par la plateforme de génération peptidique.

Les Acteurs :

-Utilisateur : L'acteur principal (chercheur ou biologiste) qui interagit avec l'interface pour définir les paramètres et consulter les résultats.

-Moteur d'exploration intelligent(Microservice) : Un acteur secondaire (système automatisé) qui exécute l'algorithme DFS et les calculs heuristiques pour générer les séquences optimales et l'historique.

Description des Cas d'Utilisation :

Le système offre huit fonctionnalités principales organisées comme suit :

S'authentifier : Permet à l'utilisateur d'accéder à son espace personnel de manière sécurisée (via JWT).

Saisir les contraintes : L'utilisateur définit les paramètres physico-chimiques cibles (masse, hydrophobicité, etc.).

Lancer la génération : Déclenche le moteur IA pour rechercher la séquence répondant aux contraintes saisies.

Consulter les résultats : Permet de visualiser la séquence générée ainsi que l'historique de décision (XAI).

Comparer les peptides : Fonctionnalité permettant d'analyser les différences de performance entre plusieurs séquences générées.

Sauvegarder les séquences : Persistance du résultat final dans la base de données pour une utilisation ultérieure.

Supprimer une séquence : Permet à l'utilisateur de nettoyer son historique en effaçant les résultats non pertinents.

Se déconnecter : Assure la fermeture sécurisée de la session de l'utilisateur.

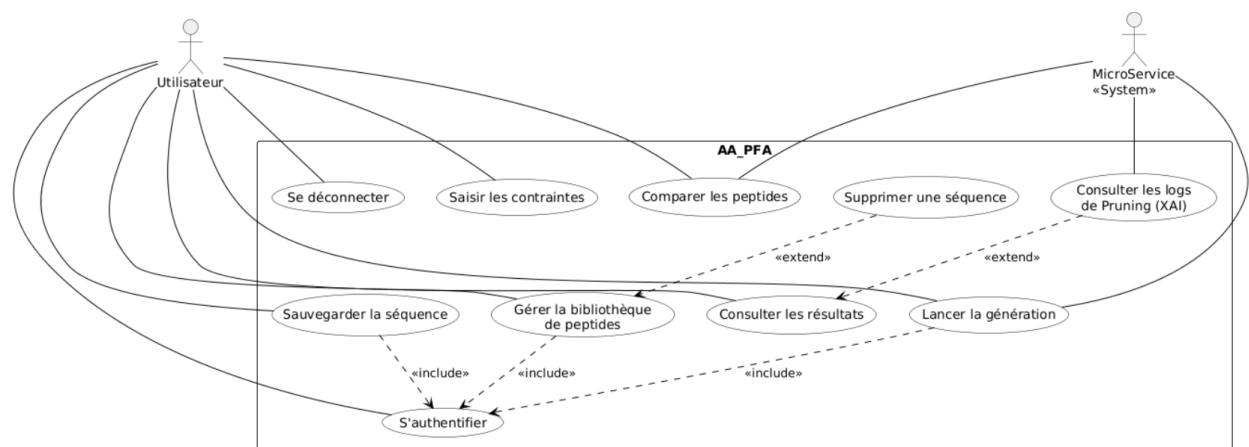


Figure 3:Diagramme de cas d'utilisation

IV ENVIRONNEMENT DE REALISATION

Cette section est consacrée pour les ressources techniques pour le développement :

4.1 Environnement Logiciel (Stack Technique)

Une sélection de technologies modernes choisies :

Langages & Core :

- Java 17 (LTS) : Pivot central pour la gestion multi-threadée et la sécurité.
- Python 3 : Moteur de calcul et génération
- TypeScript / HTML5 / CSS3 /Bootstrap : Base du frontend Angular

Framework & Serveurs :

- Spring Boot 4 : Orchestrateur de services (Spring Security, Spring Data JPA).
- Flask API : Micro-service Python assurant le Data Streaming (yield).
- Angular 17 : Framework de présentation pour une UI réactive.

Bibliothèques & Sécurité :

- NumPy : Calcul de similarité via l'intégration numérique (np.trapz).
- JWT (Java JSON Web Token) : Implémentation du protocole d'authentification sans état.
- Chart.js : Rendu dynamique des profils d'hydropathie

4.2 ENVIRONNEMENT MATERIEL & TESTS

Outils de Diagnostic : Postman (Validation des contrats d'interface REST)

Système de Gestion : MySQL pour la persistance relationnelle des séquences et des logs de justification.

V.PLANIFICATION, METHODOLOGIE SCRUM :

nous avons adopté une approche structurée basée sur les principes de l'agilité

5.1 Méthodologie Agile (Scrum)

Incrémentation : Chaque sprint a délivré une brique fonctionnelle (le moteur, puis la sécurité, puis l'interface).

Approche Itérative : Ajuster les coefficients d'élagage après chaque test de performance.

Feedback Constant : Facilite l'intégration des contraintes biochimiques et mathématiques au fur et à mesure de l'avancement.

5.2 Définition des sprints

Sprint	Objectifs et Livrables attendues
Sprint 1	Implémentation du moteur python, DFS, fonctions d'élagage, calcul AUC et vérification de la logique des méthodes
Sprint2	Configuration de Spring Boot, mise en place de Spring Security et JWT, création du schéma MySql
Sprint 3	Développement du front end Angular, Design ergonomique, intégration des courbes de profils des propriétés et streaming des données pour XAI
Sprint 4	Re-optimisation du module python, Test d'intégrations et finalisation de la documentation du projet